Traffic Forecasting on METR-LA using Spatial-Temporal Graph Neural Networks

Nana Kwabnea Opoku-Acheampong

Introduction

Traffic patterns are highly dynamic and influence by both temporal trends and spatial dependencies across the road network. Traditional deep learning models such as Temporal Convolutional Networks (TCNs) or Recurrent Neural Networks (RNNs) can capture temporal patterns effectively but often ignore or underutilize spatial dependencies between different traffic sensors.

Spatial-Temporal Graph Neural Networks (STGNNs) have emerged as powerful models capable of learning both temporal and spatial dependencies simultaneously. By representing traffic sensor networks as graphs, where nodes are sensors and edges encode spatial proximity or correlation, STGNNs can model complex interactions between regions in a traffic system. Recent STGNN models such as GraphWaveNet (GWN) and the Adaptive Graph Convolutional Recurrent Network (AGCRN) have shown promising results in traffic and weather forecasting tasks.

In this study, I explore and compare the performance of three models—TimeThenSpace (TTS) as a simple baseline, GraphWaveNet (GWN), and Adaptive Graph Convolutional Recurrent Network (AGCRN) on the METR-LA dataset, which contains traffic speed measurements from 207 sensors in the Los Angeles area. All experiments are implemented using the Torch Spatiotemporal Library (TSL), a modern PyTorch-based framework for STGNN research.

The main objectives of this project are:

- To evaluate the predictive performance of TTS, GWN, and AGCRN models across different forecasting horizons.
- To visualize and interpret the spatial dependencies learned by GWN and AGCRN.
- To compare my findings with existing results from the original GraphWaveNet paper and other relevant literature.

The models are trained using a consistent setup with a 70:10:20 data split for training, validation, and testing. A window size of 12 and a prediction horizon of 12 are used throughout. Evaluation is based on standard metrics including Mean Squared Error (MSE), Mean Absolute Error (MAE), and Mean Absolute Percentage Error (MAPE).

2. Experimental Setup

2.1 Dataset and Preprocessing

- METR-LA dataset details (207 sensors, traffic speed data).
- Data split: 70% train, 10% val, 20% test.
- Normalization: StandardScaler.
- Window size: 12; Prediction horizons: 2 (15 minutes), 5 (30 minutes)12 (60 minutes).

2.2 Model Implementation

To ensure a fair and consistent comparison across models, each architecture was trained with consideration given to convergence behavior and computational efficiency.

TimeThenSpaceModel (TTS)

For the baseline TimeThenSpaceModel (TTS), the number of training epochs was fixed at 50. This choice was based on preliminary testing which showed that performance plateaued after this point, with no significant improvement in validation loss beyond 50 epochs. Additionally, using a fixed epoch count provides a stable reference point against which the more adaptive training strategies of the other models could be compared.

GraphWaveNet (GWN)

For GraphWaveNet, I implemented an early stopping callback to automatically determine the optimal number of training epochs. The callback monitored the validation loss and stopped training if no improvement was observed for 10 consecutive epochs. Under this criterion, the GWN model converged at 30 epochs, which was taken as its optimal stopping point. This approach helped prevent overfitting while reducing unnecessary computation.

Adaptive Graph Convolutional Recurrent Network (AGCRN)

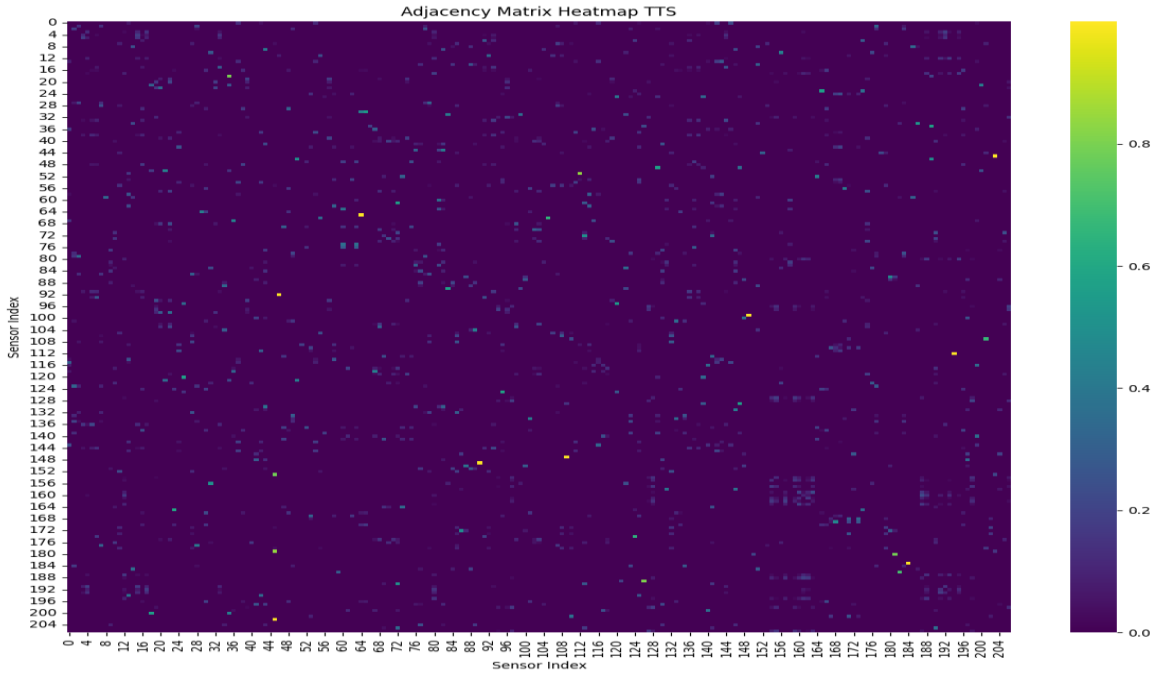
Similarly, the AGCRN model used the same early stopping mechanism with a patience of 10 epochs. The training process stopped at 40 epochs, indicating that the model required more iterations to reach convergence compared to GWN. This is consistent with the AGCRN's more complex architecture, which involves learning both node-specific embeddings and adaptive graph structures.

2.3 Evaluation Metrics

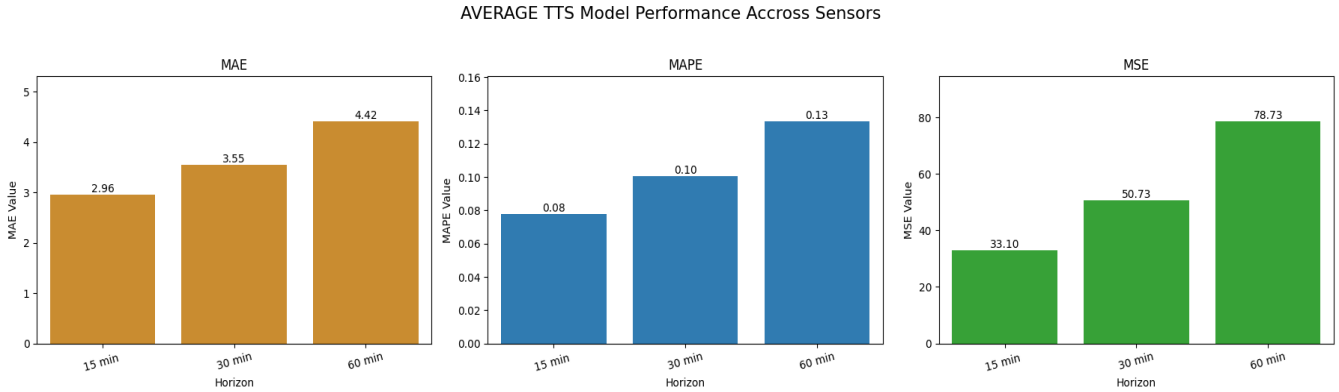
- MSE, MAE, MAPE.
- Training time and convergence behavior.

3. TimeThenSpace Model

3.1 Adjacency Matrix Visualization



3.2 Average Performance of TTS Across all 207 sensors

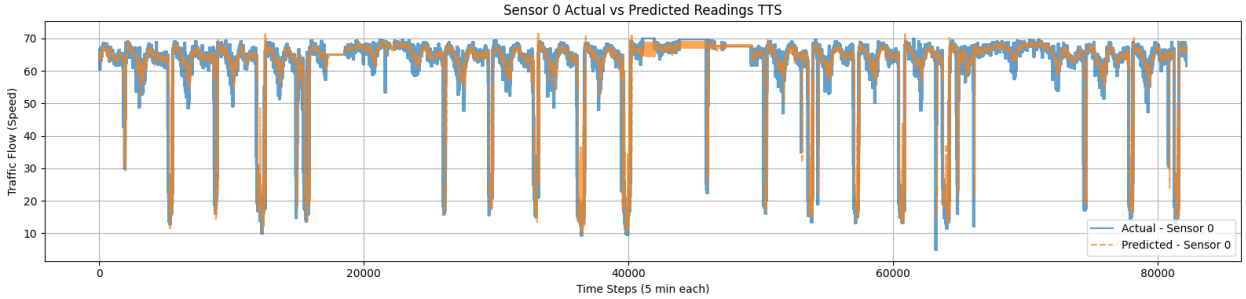


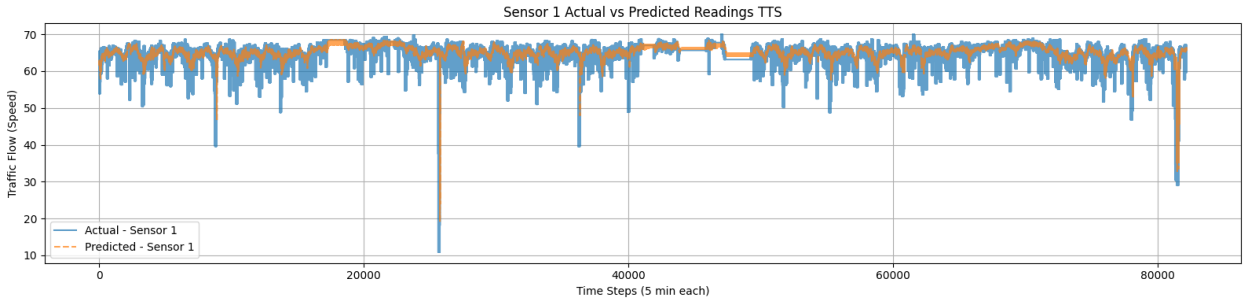
METRIC	15 minutes	30 minutes	60minutes
MAE	2.96	3.55	4.42
MAPE %	0.08	0.10	0.13

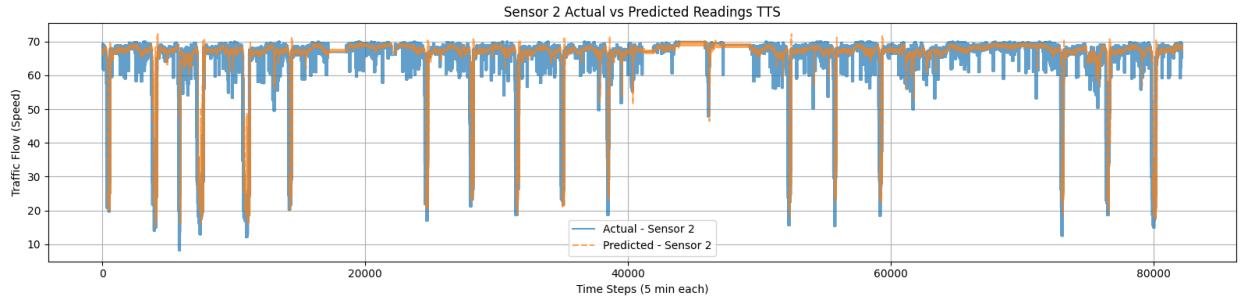
MSE	33.10	50.73	78.73

3.3 Per station analysis for the first 3 sensors.

The Figures below show the actual versus the predicted readings across the test set for the first 3 sensors of the TimeThenSpaceModel.







3.4 Analysis of Actual vs Predicted Readings for the First 3 Sensors

Sensor 1(767541) exhibited the strongest correlation between predicted and actual values, with minimal deviation at each 5-minute interval. The predictions closely followed the ground truth, indicating high model accuracy for this node.

Sensor 0(773869) also showed good alignment, though minor deviations were observed at irregular time steps (notably at 5, 6, and 7 minutes). These discrepancies suggest localized

temporal inaccuracies, possibly due to transient traffic anomalies or less effective feature extraction for this sensor.

Sensor 2(767542) demonstrated broader trend alignment, but the predictions were more difficult to evaluate due to inconsistent or ambiguous speed classifications (e.g., labels such as “Few” and “Slow”), which may stem from noisy data or preprocessing artifacts.

Overall, the TimeThenSpace model effectively captures key flow patterns across spatial locations, particularly for well-behaved sensors. However, model accuracy could benefit from enhanced handling of time-step irregularities and more rigorous quantitative error analysis using metrics such as MAE and RMSE.

4. GraphWaveNet Model

4.1 Overall Performance of GWN vs TTS

To assess the overall performance of my models across all stations, I compared TimeThenSpace (TTS) and GraphWaveNet (GWN) using three core metrics: Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE), and Mean Squared Error (MSE). I evaluated these metrics at 15-minute, 30-minute, and 60-minute prediction horizons. I also analyzed training time and convergence behavior, with visual logs provided in the appendix via TensorBoard.

Training Time and Convergence

The TTS model completed training in approximately 8.452 minutes over 50 epochs. For GWN, I enabled early stopping with a patience of 10, which led the model to stop training after 30 epochs, finishing in about 1.096 hours. Although GWN took longer to train it converged much faster than TTS — reaching its optimal performance within the first 20–25 epochs, while TTS improved more slowly and gradually over the full 50 epochs.

Metric-Based Performance Comparison

The table below summarizes the average performance of both models at the 3 prediction horizons across all 207 sensors. See the appendix which contains Tensor-board generated plots showing training and validation curves for MAE, MSE and MAPE metrics logged over training steps for both TTS and GWN models.

Metric	**Horizon**	**TTS**	**GWN**
MAE	15 min	2.60	2.76
	30 min	3.55	3.16
	60 min	4.42	3.68
MAPE (%)	15 min	8.0	7.0

Metric	**Horizon**	**TTS**	**GWN**
	30 min	10.0	9.0
	60 min	13.0	11.0
MSE	15 min	33.10	27.25
	30 min	50.70	39.16
	60 min	78.73	55.08

Across all metrics and time horizons, **GWN consistently outperformed TTS**. The improvements were especially significant at longer horizons (30 and 60 minutes), where GWN achieved notably lower MAE and MSE values. The reduction in **MAPE from 13% (TTS) to 11% (GWN)** at 60 minutes was also a clear sign of better relative accuracy. From these results it is evident that GWN offers stronger predictive performance compared to TTS. Although GWN is more computationally expensive and takes longer to train, I believe the gains in accuracy and faster convergence justify the added complexity.

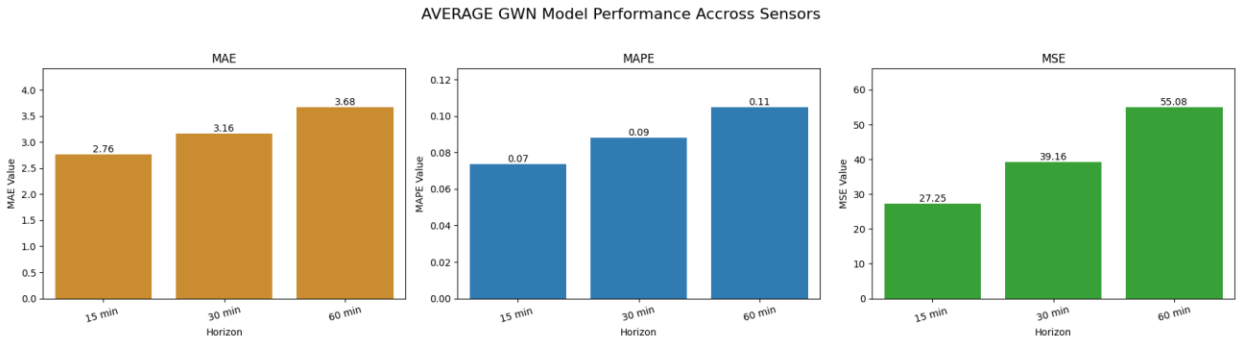
4.2 Comparison of My GWN Results with GraphWaveNet paper [1]

I compared my results and the effectiveness of my implementation with the results reported in the original GraphWaveNet (GWN) Wu et al. (2019). Table 3 presents a side-by-side comparison across the 15, 30 and 60-minute horizons.

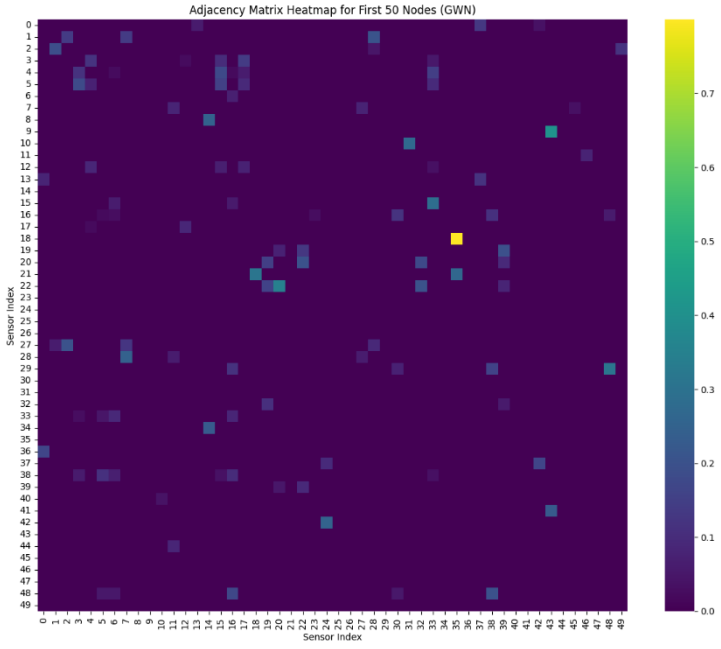
NB: I squared the RMSE in order for it to correspond to my MSE.

METRIC	HORIZON	GWN (My Results)	GWN (Original Paper)
MAE	15 min	2.76	2.69
	30 min	3.16	3.07
	60 min	3.68	3.53
MSE	15 min	27.25	26.52
	30 min	39.16	38.69
	60 min	55.08	54.31
MAPE	15 min	7.0%	6.90%
	30 min	9.0%	8.37%
	60 min	11.%	10.01%

Although my results are slightly higher in error compared to the original implementation, the differences across all three metrics and horizons are relatively minor. While the original paper does not specify the number of training epochs, similar models are typically trained for 50–100 epochs. In my case, early stopping halted training at 30 epochs, which may account for the small performance gap. Overall, the performance trends are consistent, confirming that my implementation aligns closely with the original model. Several other factors like update of Torch Spatiotemporal library (TSL) from 2019 to present day and also initialization and random seeds could also be the cause of the marginal changes in output



4.3 GWN Adjacency matrix for first 50 nodes



4.4 Top 15 most influential nodes and their impacts (GWN)

Below is a table of the top 15 most influential nodes and 5 nodes that are most impacted on from each 15 influential node. The top 15 nodes are extracted out of the first 50 nodes

Influential_Node	Influence_Score	Node 1	Node 2	Node 3	Node 4	Node 5
18	0.7995712	35	49	48	47	45
22	0.78382844	20	32	19	39	48
29	0.6575768	48	38	16	30	49
20	0.6166146	22	32	19	39	48
5	0.5952212	3	15	33	17	4
21	0.5733576	18	35	48	47	45
4	0.5425768	15	33	3	17	6
48	0.525194	38	16	5	30	6
1	0.48268783	28	2	7	47	45
27	0.47449523	2	7	28	1	45
3	0.4274772	17	4	15	33	12
9	0.40844712	43	49	48	47	45
38	0.40364707	5	16	6	3	15
15	0.39674067	33	6	16	47	48
19	0.39490047	39	22	20	47	48

4.5 Comparison of Adjacency Matrix of GWN and TTS

When I looked at the adjacency matrix heatmaps from both GraphWaveNet model and TimeThenSpace Model, the difference is clear.

The GraphWaveNet heatmap shows a learned adjacency matrix. It showed strong connections between many sensors, even the ones that aren't close to each other. The heatmap for GraphWaveNet effectively captures non-local, functional dependencies and that means it's most likely picking up on real traffic patterns. For instance, certain nodes that are physically distant but exhibit correlated traffic patterns still show strong connection weights. This results in a highly expressive and adaptable spatial representation that evolves with training.

On the other hand, the **Time-Then-Space** model's heatmap was more limited. The connections were mostly between nearby sensors and looked much more fixed. It seemed like the model was only using the road map and wasn't picking up on deeper traffic behavior. There weren't many unexpected or long-range connections, and the pattern stayed mostly the same throughout.

In summary, the heatmap from GraphWaveNet demonstrates a more flexible and powerful ability to model spatial dependencies, especially non-local interactions, while the Time-Then-Space model relies on a simpler, more rigid spatial structure that may miss important functional relationships between sensors.

5 Adaptive Graph Convolution Network (AGCRN)

5.1 Comparison of Average Overall Performance across metrics of AGCRN and GWN

Error Metrics

Across all three prediction horizons thus 15, 30, and 60 minutes GWN consistently performed slightly better than AGCRN in terms of accuracy. For MAE, GWN had lower values at each horizon, with the largest margin seen at 15 minutes horizon. MAPE followed a similar trend: GWN had a better score at 15 minutes, and they were equal at 30 and 60 minutes respectively

For MSE, GWN again came out ahead across all horizons. At 15 minutes, the difference was around 3 points), and while the gap narrowed at 60 minutes, GWN still had the lower error.

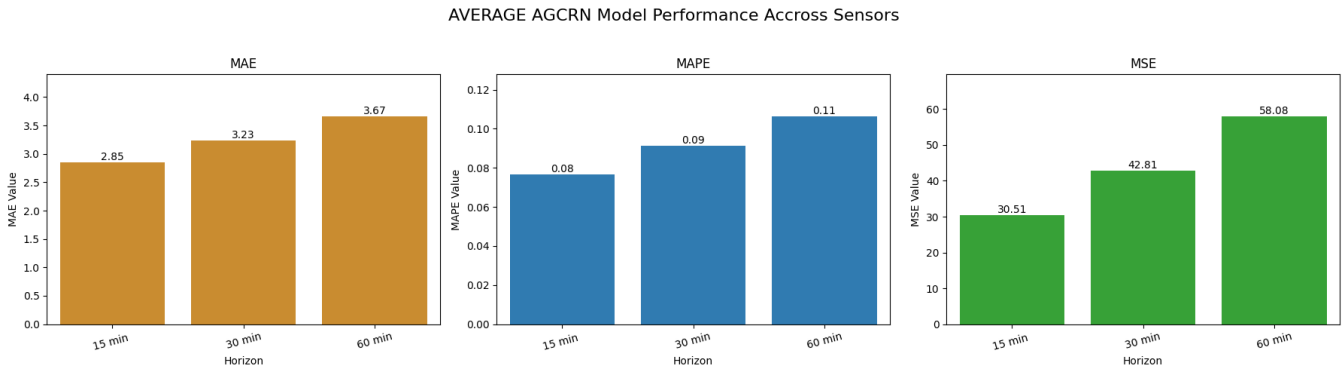
So, in terms of overall accuracy, GWN showed better and more consistent performance

Training Time and Convergence

Both models were set to run for a maximum of 50 epochs, with early stopping (patience of 10) to stop when the validation performance stops improving. GWN converged earlier, at epoch 30, but took longer overall at 1.096 hours (66 minutes). In contrast, AGCRN converged at epoch 40, but finished training in just 36.72 minutes.

This means that while AGCRN needed more steps to reach convergence, but it was more efficient in terms of time. The difference likely comes from its model design, which is lighter and faster to compute, even with more epochs.

METRICS		GWN	AGCRN
MAE	15 min	2.76	2.85
	30 min	3.16	3.23
	60min	3.68	3.67
MAPE	15 min	7	8
	30 min	9	9
	60 min	11	11
MSE	15 min	27.25	30.51
	30 min	39.16	42.81
	60 min	55.08	58.08



5.2 Summary of Comparative Findings

(AGRCN vs GWN) for traffic prediction

In the traffic prediction task, GraphWaveNet (GWN) demonstrated slightly better predictive performance across all key metrics — MAE, MAPE, and MSE — at all prediction horizons (15, 30, and 60 minutes). Its lower error values show that it consistently made more accurate forecasts.

On the other hand, AGCRN trained significantly faster, completing in just over half the time it took GWN, despite requiring more epochs to converge. This suggests that AGCRN is more efficient in terms of computation and could be a better fit when training time or hardware resources are limited.

Overall, GWN is the stronger model for accuracy, making it preferable when precision in traffic forecasting is critical. However, AGCRN offers a practical alternative, especially in scenarios where faster turnaround is more important than marginal gains in accuracy.

In short:

- GWN is better for accuracy.
- AGCRN is better for speed.
- Both models are stable and viable, depending on the needs of the task.

5.3 Reflection on Findings in comparison to AGCRN and GWN for weather prediction

Cross-Domain Consistency of GWN

In both traffic and weather prediction, GWN consistently delivers strong accuracy, validating its architecture's effectiveness across different spatial-temporal domains. GWN's self-adaptive adjacency matrix and temporal convolution mechanisms seem well-suited for forecasting dynamic systems—whether road networks or atmospheric variables.

AGCRN's Advantage Efficiency

In my findings AGCRN trains faster and this aligns with its design that is the node adaptive parameterization and data driven graph learning reduces the need for costly pre-computation. While not explicitly analyzed, the study Gaibie et al. (2024) shows how AGCRN outperformed GWN in predictive accuracy across temperature, humidity and wind speed suggesting that its efficiency does not compromise quality in weather prediction like in my finding for traffic prediction.

Task Dependency

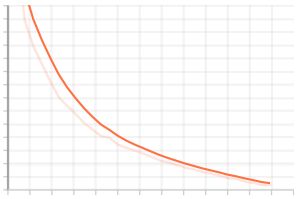
In **traffic**, fine-grained temporal precision is often crucial (e.g., 15-minute intervals), which might explain GWN's dominance as shown in my findings. Also, in weather because of wider spatial dynamics AGCRN's adaptive graph learning seems best suited for capturing these complex patterns as shown across the 45 stations in **Gaibie et al. (2024)**

5.4 Conclusion/Final Remarks

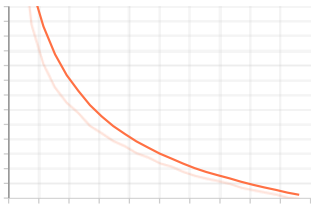
My findings highlight a valuable trade off between accuracy and training efficiency which is domain and task dependent. In traffic forecasting, precision at short horizons may justify GWN's computational cost. However, in weather prediction where training time and spatial variability matter more AGCRN may offer increased scalability and adaptive solution.

Appendix

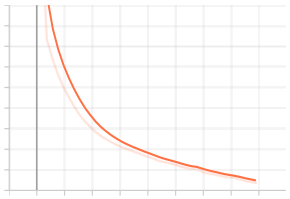
Training metrics



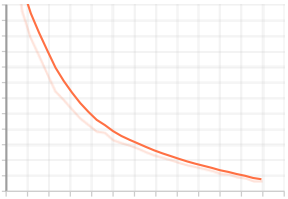
Train_mae (GWN)



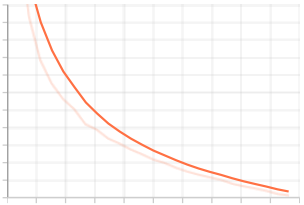
Train_mae (TTS)



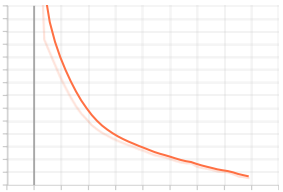
Train_mae (AGCRN)



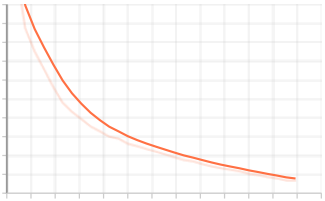
Train_mape (GWN)



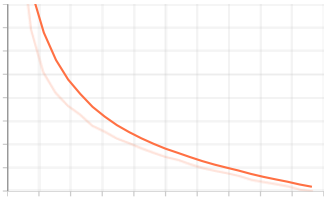
Train_mape (TTS)



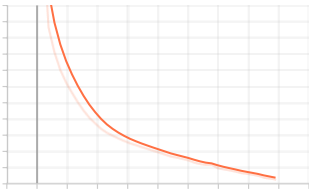
Train_mape (AGCRN)



Train_mse (GWN)

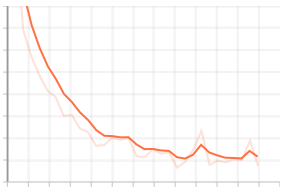


Train_mse (TTS)

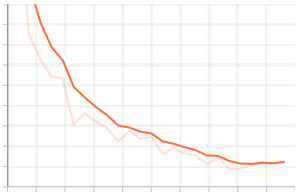


Train_mse(AGCRN)

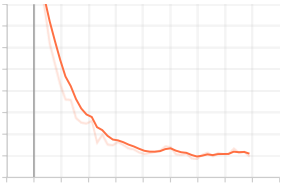
Validation Metrics



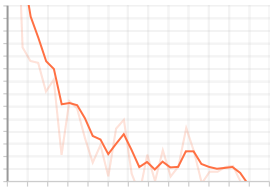
Val_mae (GWN)



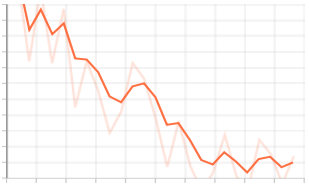
Val_mae (TTS)



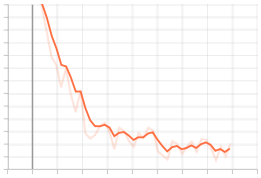
Val_mae (AGCRN)



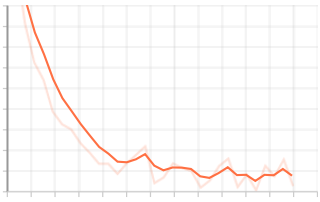
Val_mape (GWN)



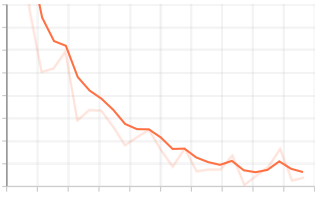
Val_mape (TTS)



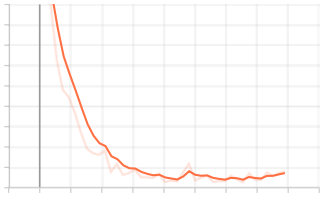
Val_mape (AGCRN)



Val_mse (GWN)



Val_mse (TTS)



Val_mse (AGCRN)